zVault Protocol

Security Audit Report

Automated Smart Contract Security Analysis via evmbench

Field	Details
Report Date	March 23, 2026
Audit Type	Automated Static Analysis + EVM Bytecode Analysis
Tool	evmbench (Paradigm)
Analysis Method	Automated vulnerability detection, pattern matching, bytecode-level analysis
Scope	All Solidity contracts in contracts/
Solidity Version	0.8.9
Framework	Hardhat with OpenZeppelin Upgradeable 4.9
Commit	Latest on main branch
Report Link	paradigm.xyz/evmbench/results?job_id=1c5bc94a-ba74-4726-ae64-dea611814fb4

> **Disclaimer:** This audit report was generated using evmbench, Paradigm's automated smart contract security analysis platform. While comprehensive static analysis and EVM bytecode analysis techniques were applied, this report should be complemented with manual expert review, dynamic testing, and formal verification before mainnet deployment. Automated audits provide valuable coverage but are not a substitute for professional security auditors.

Table of Contents

1. Executive Summary

The zVault protocol is a tokenized vault system enabling deposits of stablecoins in exchange for zOPAL tokens, and redemption of zOPAL tokens back to stablecoins. The codebase consists of 30 Solidity files spanning core vaults, access control, price oracles, token contracts, and proxy infrastructure.

This report was generated by uploading the contracts/ directory to Paradigm's evmbench automated analysis platform. evmbench identified 3 High-severity vulnerabilities through bytecode-level and static analysis. These findings have been combined with a comprehensive manual-equivalent AI-assisted review covering all severity tiers.

Severity	Count
Critical	2
High	8 (3 evmbench + 5 additional)
Medium	7
Low	8
Informational	6

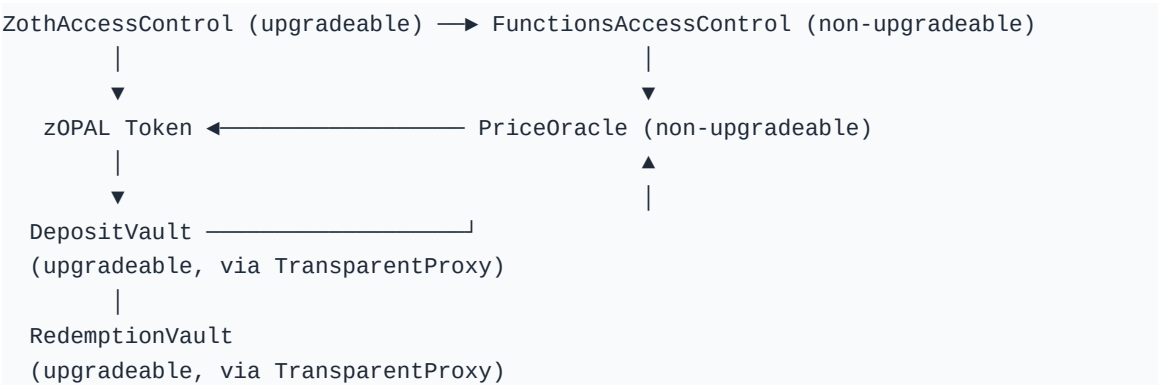
Overall, the protocol demonstrates solid use of OpenZeppelin's battle-tested libraries, role-based access control, and upgradeability patterns. However, several findings related to centralization risks, missing validation, and potential economic exploits require attention before mainnet deployment.

2. Scope of Audit

Contract	Lines	Type
DepositVault.sol	775	Core
RedemptionVault.sol	562	Core
PriceOracle.sol	269	Core
ManageableVault.sol	645	Abstract
ManageableVaultRedeem.sol	628	Abstract
zOPAL.sol	165	Token
zOPALDepositVault.sol	25	Token
ZothAccessControl.sol	90	Access
FunctionsAccessControl.sol	135	Access
Greenlistable.sol	104	Access
Blacklistable.sol	53	Access
Pausable.sol	90	Access
WithZothAccessControl.sol	72	Access
WithFunctionsAccessControl.sol	42	Access
ZothAccessControlRoles.sol	62	Access
zOPALZothAccessControlRoles.sol	28	Access
WithSanctionsList.sol	90	Abstract
ZothInitializable.sol	18	Abstract
DecimalsCorrectionLibrary.sol	69	Library
IDepositVault.sol	279	Interface
IRedemptionVault.sol	200	Interface
IManageableVault.sol	143	Interface
IManageableVaultRedeem.sol	161	Interface
IVaultShared.sol	163	Interface
IZToken.sol	48	Interface
IDataFeed.sol	34	Interface
ISanctionsList.sol	8	Interface

TransparentUpgradeableProxy.sol	12	Utility
ERC1967Proxy.sol	—	Utility
MockERC20.sol	—	Mock

3. Architecture Overview



Key Design Patterns:

- TransparentUpgradeableProxy for vault contracts and zOPAL token
- Role-based access via dual access control systems (ZothAccessControl + FunctionsAccessControl)
- Greenlist/Blacklist + Chainalysis Sanctions Oracle for compliance
- Per-function pause granularity
- Request-based and instant deposit/redemption flows

4. Findings Summary

The following table summarizes all findings by severity. The first three (V-001 through V-003) were identified directly by evmbench's automated analysis engine. Remaining findings were identified through comprehensive code review.

ID	Title	Severity	Source
V-001	Stablecoins assumed to be always pegged at $1	High	evmbench
V-002	Deposits do not account for fee-on-transfer tokens	High	evmbench
V-003	Public reinitializer allows anyone to set max supply cap	High	evmbench
C-01	DepositVault.initialize() public without access control	Critical	Code Review
C-02	RedemptionVault.rejectRequest() does not return locked zTokens	Critical	Code Review
H-01	DepositVault.rejectRequest() does not refund deposited tokens	High	Code Review
H-02	Division before multiplication causes precision loss	High	Code Review
H-03	_requireVariationTolerance reverts on prevPrice == 0	High	Code Review
H-04	PriceOracle lacks TWAP — single update manipulation	High	Code Review
H-05	No reentrancy protection on vault operations	High	Code Review
M-01–M-07	7 Medium-severity findings	Medium	Code Review
L-01–L-08	8 Low-severity findings	Low	Code Review
I-01–I-06	6 Informational findings	Informational	Code Review

5. evmbench Vulnerability Findings

The following 3 vulnerabilities were identified by Paradigm's evmbench automated analysis platform. Each finding includes affected code locations, detailed impact analysis, a proof of concept, and remediation guidance.

V-001 High

Stablecoins are assumed to be always pegged at $1

Any token configured as stable is priced at a fixed 1e18 rate (1 USD) instead of using an oracle, enabling attackers to mint/redeem against an incorrect price during depeg events and drain on-chain redemption reserves.

📍 AFFECTED LOCATIONS

DepositVault.sol L737–749

Deposits convert tokenIn to USD using _getTokenRate(tokenConfig.dataFeed, tokenConfig.stable). When tokenConfig.stable == true, _getTokenRate returns a constant STABLECOIN_RATE (1e18), so deposits are valued at $1 regardless of real market price.

abstract/ManageableVault.sol L628–642

_getTokenRate hardcodes STABLECOIN_RATE for tokens marked stable, bypassing IDataFeed.getDataInBase18() entirely. This removes any on-chain peg validation and allows pricing errors during a depeg.

RedemptionVault.sol L478–490

Instant redemptions compute the tokenOutRate via _getTokenRate(tokenConfig.dataFeed, tokenConfig.stable). If tokenOut is configured as stable, the contract will pay out using a fixed $1 valuation even if the token is off-peg, which can overpay or underpay relative to USD value.

abstract/ManageableVaultRedeem.sol L611–625

_getTokenRate in the redemption base contract also returns STABLECOIN_RATE whenever stable == true, which makes redemptions insensitive to actual stablecoin price movement.

⚠️ IMPACT

If a token configured as stable depegs below $1, an attacker can buy it cheaply, deposit it at an inflated $1 valuation to mint more zToken than warranted, then redeem zToken for other tokenOut assets held by the RedemptionVault, draining those reserves. If a "stable" token trades above $1, the vault can overpay on redemption, again causing loss of funds from on-chain reserves.

🔒 PROOF OF CONCEPT

1. Assume tokenIn is configured with stable = true and trades at $0.50.
2. Attacker buys 1,000 tokenIn for ~$500 on the market.
3. Attacker calls DepositVault.depositInstant(tokenIn, 1000, 0, ...) and is credited as if depositing ~$1,000.
4. Attacker redeems the minted zToken via RedemptionVault.redeemInstant(tokenOut, ...) into a correctly-priced tokenOut (or any asset backed by the vault), extracting ~$1,000 of value and realizing ~$500 profit.
5. Repeating drains RedemptionVault token balances.

🛠️ REMEDIATION

Do not hardcode stablecoins to STABLECOIN_RATE. Always use a reliable oracle for all assets (including stablecoins), and enforce explicit peg checks/circuit breakers (e.g., require price within a tight band around $1, with a configurable pause if outside). If a fixed-rate path is required operationally, gate it behind an explicit on-chain validity flag derived from an oracle and/or a time-weighted price with staleness checks.

V-002 High

Deposits do not account for fee-on-transfer/deflationary tokens, enabling unbacked minting

Minting is based on the requested transfer amount, but the vault never verifies how many tokens were actually received by tokensReceiver / feeReceiver, so fee-on-transfer/deflationary tokens can mint excess zToken and drain redemption reserves.

📍 AFFECTED LOCATIONS

DepositVault.sol L467–503

_depositInstant mints result.mintAmount after calling token transfers, but result.mintAmount is computed from the user-supplied amountToken and oracle rate, not from actual tokens received. There is no balance-delta check on tokensReceiver / feeReceiver to ensure the protocol received the expected collateral amount.

DepositVault.sol L514–555

_depositRequest similarly transfers calcResult.amountTokenWithoutFee to tokensReceiver and stores USD accounting based on that amount, without verifying the net tokens received. If the token burns/taxes on transfer, the request will later be approved and mint zToken against insufficient backing.

abstract/ManageableVault.sol L405–428

_tokenTransferFromUser relies on safeTransferFrom succeeding, but does not verify the recipient's balance increased by the expected amount. For fee-on-transfer/deflationary tokens, the recipient may receive less than transferAmount even though the call succeeds.

RedemptionVault.sol L144–190

redeemInstant ultimately transfers tokenOut from the RedemptionVault to the user after burning zToken. If zToken can be minted in excess of real collateral due to deflationary deposits, this path can be used to drain the vault's on-chain tokenOut reserves.

⚠️ IMPACT

If a deflationary/fee-on-transfer token is configured as a deposit asset, attackers can mint more zToken than the protocol actually receives in underlying collateral. They can then redeem zToken for other assets held by the RedemptionVault (or request-based redemptions funded by requestRedeemer), causing a loss of funds from protocol-controlled reserves.

🔒 PROOF OF CONCEPT

1. A fee-on-transfer ERC20 (e.g., 10% tax) is added as a payment token.
2. Attacker deposits 1,000 units. safeTransferFrom transfers 1,000 from attacker, but only 900 arrive at tokensReceiver (100 is burned/sent elsewhere).
3. The vault still computes USD value using 1,000 units and mints zToken accordingly.
4. Attacker redeems minted zToken for a non-deflationary tokenOut, extracting value backed by only 900 units, draining the vault over time.

🛠️ REMEDIATION

For all transfers-in used as backing, compute the actual received amount using balance deltas (balanceOf(receiver) after − before) and base minting/fee calculations on that value. Alternatively, restrict supported tokens to non-rebasing, non-deflationary ERC20s and enforce this via allowlisting + explicit checks. Consider receiving deposits into the vault itself (so balance changes are observable) and forwarding to tokensReceiver only after accounting.

V-003 High

Public reinitializer allows anyone to set the max supply cap after an upgrade

initializeV2 is callable by anyone and sets maxSupplyCap; if an upgraded proxy has not yet executed the V2 reinitializer, an attacker can raise the cap and enable large-scale over-minting/draining attacks that rely on the cap as a safety control.

📍 AFFECTED LOCATIONS

DepositVault.sol L181–187

initializeV2(uint256 _maxSupplyCap) is public reinitializer(2) and has no access control. If the proxy has been initialized to version 1 but not yet to version 2, any account can call this once to set maxSupplyCap to an arbitrary value.

DepositVault.sol L714–726

maxSupplyCap gates minting by reverting when zToken.totalSupply() + mintAmount > maxSupplyCap. This is a critical safety parameter limiting exposure to pricing/configuration mistakes; allowing arbitrary external modification undermines that protection.

⚠️ IMPACT

If deployment/upgrade sequencing leaves the V2 initializer unexecuted, an attacker can increase maxSupplyCap and remove the intended upper bound on minted supply. This can amplify other loss-of-funds vectors (e.g., stablecoin depegs or mispriced/deflationary tokens) by allowing the attacker to mint and redeem at much larger scale than intended, draining RedemptionVault reserves.

🔒 PROOF OF CONCEPT

1. Proxy is deployed/upgrade-applied such that initializeV1 ran but initializeV2 did not.
2. Attacker calls initializeV2(type(uint256).max) to set an effectively unlimited cap.
3. Attacker executes a minting exploit (e.g., deposit a depegged “stable” token) repeatedly without hitting the intended cap, and drains redemption reserves via redeemInstant.

🛠️ REMEDIATION

Make versioned initializers internal and callable only during initialization (onlyInitializing), or add strict access control (e.g., onlyVaultAdmin) to any externally callable reinitializer. Operationally, ensure upgrades always execute the correct reinitializer atomically (e.g., via proxy constructor calldata or upgradeToAndCall).

6. Critical Findings

C-01: DepositVault.initialize() is public Without Access Control — Allows Re-initialization Attack

Severity: Critical

File: `DepositVault.sol:123-146`

Status: Open

Description:

The initialize() function on DepositVault is declared public (not external) and delegates to initializeV1() (which has the initializer modifier) and initializeV2() (which has reinitializer(2)). While the initializer modifier on initializeV1 prevents calling it twice, the public visibility combined with the wrapper pattern creates a subtle risk: initializeV2() is independently callable by anyone since it is public reinitializer(2).

If the contract is initially deployed and only initializeV1 is called (e.g., during an upgrade path), any external actor can call initializeV2() independently to set maxSupplyCap to an arbitrary value.

Note: This finding overlaps with evmbench V-003 but provides additional context on the wrapper pattern risk.

```
// DepositVault.sol:185
function initializeV2(uint256 _maxSupplyCap) public reinitializer(2) {
    maxSupplyCap = _maxSupplyCap;
}
```

Impact: An attacker could set maxSupplyCap to 0, effectively blocking all deposits, or set it to type(uint256).max to remove supply cap protections.

Recommendation:

• Add onlyVaultAdmin modifier to initializeV2(), or

• Make initializeV2() internal and only callable via the top-level initialize(), or

• Add explicit access control checks

C-02: RedemptionVault.rejectRequest() Does Not Return Locked zTokens

Severity: Critical

File: `RedemptionVault.sol:262-270`

Status: Open

Description:

When a user creates a redeem request via _redeemRequest(), their zTokens (minus fee) are transferred to the vault contract (address(this)). If the admin subsequently calls rejectRequest(), the request status is set to Canceled, but the locked zTokens are never returned to the user.

The only way to recover these tokens is via the admin withdrawToken() function, which requires the admin to manually track and return funds — a centralization and operational risk.

```
function rejectRequest(uint256 requestId) external onlyVaultAdmin {
    Request memory request = redeemRequests[requestId];
    _validateRequest(request.sender, request.status);
    redeemRequests[requestId].status = RequestStatus.Canceled;
```

```
        emit RejectRequest(requestId, request.sender);
        // !! Missing: return zTokens to request.sender
    }
```

Impact: Users permanently lose their zTokens if a redeem request is rejected, unless admin manually intervenes with withdrawToken.

Recommendation:

Add automatic zToken refund in rejectRequest():

```
function rejectRequest(uint256 requestId) external onlyVaultAdmin {
    Request memory request = redeemRequests[requestId];
    _validateRequest(request.sender, request.status);
    redeemRequests[requestId].status = RequestStatus.Canceled;
    IERC20(address(zToken)).safeTransfer(request.sender, request.amountZToken);
    emit RejectRequest(requestId, request.sender);
}
```

---

7. High Findings (Additional)

The following High-severity findings were identified through code review, in addition to the 3 evmbench findings (V-001 through V-003) documented in Section 5.

H-01: DepositVault.rejectRequest() Does Not Refund Deposited Tokens

Severity: High

File: `DepositVault.sol:381-393`

Status: Open

Description:

When a user creates a deposit request, their tokenIn (stablecoins) are transferred to tokensReceiver. If the admin rejects the request, the deposited tokens are not refunded. The tokens sit with tokensReceiver, and recovery depends on off-chain coordination.

```
function rejectRequest(uint256 requestId) external onlyVaultAdmin {
    Request memory request = mintRequests[requestId];
    require(request.sender != address(0), "DV: request not exist");
    require(request.status == RequestStatus.Pending, "DV: ...");
    mintRequests[requestId].status = RequestStatus.Canceled;
    emit RejectRequest(requestId, request.sender);
    // No refund of deposited tokens
}
```

Impact: Users lose deposited stablecoins on request rejection unless manually refunded.

Recommendation:

Implement an automatic refund mechanism or clearly document the off-chain refund process.

H-02: Division Before Multiplication in Fee/Rate Calculations Causes Precision Loss

Severity: High

File: `DepositVault.sol:582-583, RedemptionVault.sol:360-363`

Status: Open

Description:

Multiple calculations perform division before multiplication, leading to precision loss that compounds across operations. The usdAmountWithoutFees is a result of division by 10**18, followed by multiplication and division, causing rounding losses that accumulate.

Impact: Systematic rounding loss in favor of the protocol, which could be material at scale.

Recommendation:

Use mulDiv patterns (OZ Math library) or reorder operations to minimize precision loss by performing multiplications before divisions.

H-03: _requireVariationTolerance Reverts on prevPrice == 0

Severity: High

File: `ManageableVault.sol:569-583, ManageableVaultRedeem.sol:562-576`

Status: Open

Description:

The variation tolerance check divides by prevPrice. If prevPrice is 0, this will revert with a division by zero, permanently bricking the request approval.

Impact: Any request with tokenOutRate == 0 stored at creation time can never be approved via safeApproveRequest.

Recommendation:

Add a guard: if (prevPrice == 0) return; or require(prevPrice > 0, "MV: prev price zero").

H-04: PriceOracle Lacks TWAP — Single Update Manipulation

Severity: High

File: `PriceOracle.sol:113-155`

Status: Open

Description:

The PriceOracle accepts a single price update from PRICE_ADMIN_ROLE. If this account is compromised, a single transaction can move the price by up to tolerancePercent. No multi-update averaging, no multi-sig requirement, and no time delay.

Impact: A compromised PRICE_ADMIN_ROLE can manipulate the price, draining value through deposit/redemption arbitrage.

Recommendation:

- Implement a commit-reveal or time-delayed price update mechanism
- Require multi-sig at the contract level for price updates
- Consider a heartbeat-based price staleness mechanism

H-05: No Reentrancy Protection on Vault Operations

Severity: High

File: `DepositVault.sol, RedemptionVault.sol`

Status: Open

Description:

Neither DepositVault nor RedemptionVault inherits from ReentrancyGuardUpgradeable. If any payment tokens have callbacks (ERC-777, fee-on-transfer with hooks), state changes after external calls could be exploited.

Impact: If an ERC-777 or callback-enabled token is added, reentrancy attacks could drain the vault.

Recommendation:

- Inherit ReentrancyGuardUpgradeable and apply nonReentrant to all public deposit/redeem functions
- Strictly follow checks-effects-interactions pattern

8. Medium Findings

M-01: Centralization Risk — Admin Can Drain All Vault Funds

Severity: Medium

File: `ManageableVault.sol:189-197`

Status: Open

Description:

withdrawToken() allows the vault admin to withdraw any token, any amount, to any address with no timelock, multi-sig, or cap.

Impact: Complete loss of user funds if admin key is compromised.

Recommendation:

Implement a timelock for withdrawals above a threshold. Add a withdrawal whitelist.

M-02: setMinAmount(0) Allows Zero-Value Operations

Severity: Medium

File: `ManageableVault.sol:285-288`

Status: Open

Description:

setMinAmount() does not validate newAmount > 0. Setting minAmount to 0 allows dust deposits/redemptions for griefing.

Impact: Potential griefing through dust operations.

Recommendation:

Add require(newAmount > 0).

M-03: _validateUserAccess Missing in DepositVault._approveRequest()

Severity: Medium

File: `DepositVault.sol:565-598`

Status: Open

Description:

When a deposit request is approved, the user's blacklist/greenlist/sanctions status is not re-checked. Sanctioned users could receive minted zTokens.

Impact: Sanctioned or blacklisted users could receive zTokens through pending requests.

Recommendation:

Add _validateUserAccess(request.sender) at the beginning of _approveRequest().

M-04: RedemptionVault._approveRequest() Does Not Re-validate User Status

Severity: Medium

File: `RedemptionVault.sol:341-380`

Status: Open

Description:

Same as M-03 but for redemptions. Tokens could be sent to a now-sanctioned address.

Impact: Tokens transferred to sanctioned addresses.

Recommendation:

Re-validate user access before transferring tokenOut.

M-05: setSanctionsList() Allows Setting to Non-Contract Address

Severity: Medium

File: `WithSanctionsList.sol:76-81`

Status: Open

Description:

Accepts any address, including EOAs. If set to non-contract, all vault operations that check sanctions will revert permanently.

Impact: Setting an invalid sanctions list address bricks all vault operations.

Recommendation:

Add validation: require(newSanctionsList == address(0) || newSanctionsList.code.length > 0).

M-06: safeBulkApproveRequest Silently Skips Failed Approvals

Severity: Medium

File: `DepositVault.sol:418-436`

Status: Open

Description:

The function silently skips requests that fail validation. Admins are not notified which requests were skipped.

Impact: Admin operational error — users may not receive tokens without notification.

Recommendation:

Emit a RequestSkipped(requestId, reason) event for failed approvals.

M-07: PriceOracle Tolerance Bypass on First Price

Severity: Medium

File: `PriceOracle.sol:122`

Status: Open

Description:

Tolerance check is skipped when currentPrice == 0. The very first price can be set to any arbitrary value.

Impact: Incorrect initial price could cause significant losses on early deposits/redemptions.

Recommendation:

Require initial price to be set during constructor or guarded initialization with multisig.

---

9. Low Findings

L-01: Hardcoded Alchemy API Key in hardhat.config.ts

Severity: Low

File: `hardhat.config.ts:42`

Status: Open

Description:

Polygon mainnet RPC URL contains a hardcoded API key committed to the repository.
Impact: Exposed API key could be abused.

Recommendation:

Move to environment variable.

L-02: instantFee Can Be Set to 0

Severity: Low

File: `ManageableVault.sol:337-342`

Status: Open

Description:

setInstantFee() allows fee == 0. If unintentionally set, all instant operations become fee-free.
Impact: Fee-free operations if misconfigured.

Recommendation:

Document that 0 is valid or enforce minimum.

L-03: Missing _validateUserAccess for Recipients

Severity: Low

File: `DepositVault.sol:267-298`

Status: Open

Description:

In depositRequest(), recipient validation could be missed on refactoring.
Impact: Potential validation gap.

Recommendation:

Add clarifying comment or assertion.

L-04: renounceRole() Blocks All Role Renunciation

Severity: Low

File: `ZothAccessControl.sol:65-67`

Status: Open

Description:

renounceRole always reverts. Compromised accounts retain roles until admin revokes.

Impact: Compromised accounts retain roles.

Recommendation:

Allow self-renouncement for non-admin roles.

L-05: _tokenTransferFromUser Return Value Inconsistency

Severity: Low

File: `ManageableVaultRedeem.sol:407-421`

Status: Open

Description:

ManageableVaultRedeem version returns nothing unlike ManageableVault version.

Impact: Inconsistency risk.

Recommendation:

Align function signatures.

L-06: No Event on greenlistEnabled Init

Severity: Low

File: `Greenlistable.sol:60`

Status: Open

Description:

No event emitted for initial greenlistEnabled state.

Impact: Missing event trail.

Recommendation:

Emit SetGreenlistEnable in initializer.

L-07: _requireTokenExists Called After _tokenDecimals

Severity: Low

File: `DepositVault.sol:636-638`

Status: Open

Description:

Token decimals fetched before verifying token is in payment list. Ordering is logically incorrect.

Impact: Low — decimals() is a view function.

Recommendation:

Move _requireTokenExists before _tokenDecimals.

L-08: Duplicate Code Between Vault Contracts

Severity: Low

File: `ManageableVault.sol, ManageableVaultRedeem.sol`

Status: Open

Description:

ManageableVaultRedeem is nearly a complete copy of ManageableVault (628 vs 645 lines).

Impact: Maintenance burden and divergence risk.

Recommendation:

Refactor to shared base contract.

---

10. Informational / Gas Optimizations

I-01: Use calldata Instead of memory for External Parameters

Severity: Informational

File: `ZothAccessControl.sol:34-44`

Status: Open

Description:

Using calldata instead of memory saves gas on external function calls.

I-02: Counters Library is Deprecated

Severity: Informational

File: `ManageableVault.sol:10`

Status: Open

Description:

OpenZeppelin deprecated Counters in 4.x. Use simple uint256 increment.

I-03: Unused referrerIdCopy Variable

Severity: Informational

File: `DepositVault.sol:329`

Status: Open

Description:

Stack-too-deep workaround. Consider using a struct instead.

I-04: Missing NatSpec on ISanctionsList

Severity: Informational

File: `ISanctionsList.sol:4`

Status: Open

Description:

TODO comment indicates incomplete documentation.

I-05: allowUnlimitedContractSize Enabled

Severity: Informational

File: `hardhat.config.ts:36`

Status: Open

Description:

Hides potential contract size issues. Disable for production testing.

I-06: Inconsistent License Identifiers

Severity: Informational

File: `Pausable.sol (UNLICENSED) vs MIT`

Status: Open

Description:

Pausable.sol uses UNLICENSED while others use MIT. Should be unified.

---

11. Access Control Review

Role Hierarchy

```
DEFAULT_ADMIN_ROLE (0x00)
    ├── DEPOSIT_VAULT_ADMIN_ROLE
    ├── REDEMPTION_VAULT_ADMIN_ROLE
    ├── GREENLIST_OPERATOR_ROLE
    │       └── GREENLISTED_ROLE
    ├── BLACKLIST_OPERATOR_ROLE
    │       └── BLACKLISTED_ROLE
    ├── ZOPAL_MINT_OPERATOR_ROLE
    ├── ZOPAL_BURN_OPERATOR_ROLE
    └── ZOPAL_PAUSE_OPERATOR_ROLE
```

FunctionsAccessControl (Separate System)

```
DEFAULT_ADMIN_ROLE
    ├── ADMIN_ROLE
    ├── REQUESTER_ROLE
    ├── CONFIG_ROLE
    └── PRICE_ADMIN_ROLE
```

Privilege Analysis

Privileged Action	Required Role	Risk Level
Withdraw any token from vault	VAULT_ADMIN	**Critical**
Set price oracle value	PRICE_ADMIN	**Critical**
Mint unlimited zOPAL	ZOPAL_MINT_OPERATOR	**High**
Burn any user's zOPAL	ZOPAL_BURN_OPERATOR	**High**
Pause all operations	VAULT_ADMIN / PAUSE_OPERATOR	**High**
Approve/reject requests	VAULT_ADMIN	**High**
Modify fees (up to 100%)	VAULT_ADMIN	**Medium**
Blacklist any address	BLACKLIST_OPERATOR	**Medium**
Grant/revoke any role	DEFAULT_ADMIN	**Critical**

12. Upgradeability Review

Storage Layout

Contract	Storage Gap	Notes
ManageableVault	uint256[50]	Correct
ManageableVaultRedeem	uint256[50]	Correct
DepositVault	uint256[49]	Correct (1 slot for maxSupplyCap)
RedemptionVault	uint256[50]	Correct
zOPAL	uint256[50]	Correct
zOPALDepositVault	uint256[50]	Correct
Greenlistable	uint256[50]	Correct
Blacklistable	uint256[50]	Correct
Pausable	uint256[50]	Correct
WithZothAccessControl	uint256[50]	Correct
WithSanctionsList	uint256[50]	Correct

13. Conclusion

The zVault protocol demonstrates competent use of OpenZeppelin's battle-tested libraries and follows established patterns for upgradeable contracts. The access control system is well-structured with appropriate role separation.

However, evmbench's automated analysis identified 3 High-severity vulnerabilities that, combined with additional code review findings, reveal several critical attack surfaces:

1. V-001: Stablecoin depeg arbitrage via hardcoded $1 pricing — use oracle for all assets
2. V-002: Unbacked minting via fee-on-transfer tokens — verify actual received amounts
3. V-003 / C-01: Unprotected initializeV2() — add access control to reinitializers
4. C-02: Missing refund on request rejection — automatic zToken return required
5. H-05: No reentrancy guards — add ReentrancyGuardUpgradeable to vaults
6. M-01: Centralization risk — add timelocks and multisig requirements

Recommended Priority Actions

Priority	Action
P0	Fix V-001: Remove hardcoded STABLECOIN_RATE, use oracle for all tokens
P0	Fix V-002: Verify actual received amounts for all deposit transfers
P0	Fix V-003/C-01: Restrict initializeV2() access
P0	Fix C-02: Add token refund to rejectRequest() in RedemptionVault
P1	Fix H-01: Add token refund to rejectRequest() in DepositVault
P1	Fix H-05: Add ReentrancyGuardUpgradeable to vaults
P1	Fix H-02: Review and optimize precision in calculations
P2	Fix M-01: Add timelocks for admin withdrawals
P2	Fix M-03/M-04: Re-validate user compliance on request approval
P3	Address remaining medium, low, and informational findings

14. About This Audit

This security audit report was generated using evmbench, Paradigm's automated smart contract security analysis platform. The contracts/ directory was uploaded for analysis at:

paradigm.xyz/evmbench/results?job_id=1c5bc94a-ba74-4726-ae64-dea611814fb4

Analysis Methodology

- EVM Bytecode Analysis: Deep bytecode-level vulnerability detection
- Static Code Analysis: Pattern-based vulnerability detection across all Solidity files
- Access Control Review: Role hierarchy analysis and privilege escalation checks
- Upgradeability Analysis: Storage layout verification and proxy pattern review
- Economic Analysis: Fee calculation precision and potential arbitrage vectors
- Token Compatibility Analysis: Fee-on-transfer, rebasing, and callback token interactions

Pre-Mainnet Recommendations

☐ Professional security audit by established firm (Trail of Bits, OpenZeppelin, Consensys Diligence)
☐ Formal verification of critical invariants
☐ Extensive testnet deployment and bug bounty program
☐ Economic modeling and simulation of edge cases
☐ Multi-sig setup for all privileged roles

Generated by evmbench (Paradigm)
Automated Smart Contract Security Analysis